Designing with Classes

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **write** a `toString` method.
2. Be able to **write** a `static` field and accessors.
3. Be able to **define** and **use** encapsulation appropriately: knowing when a field or method should be `public` or `private`

Recap: Constructors

A constructor is invoked with the `new` keyword, e.g.

```
Book dune = new Book("Dune");
Book calc = new Book("Calculus", "James Stewart");
```

It gets matched to the appropriate constructor by its arguments, i.e.

```
public Book(String title)                  // matches dune
public Book(String title, String author)   // matches calculus
```

We say this constructor is **overloaded**.

Overloading Constructors

Only one signature can match, the following would **NOT** compile...

```
public Book(String title)
public Book(String author)
```

Why? Think about the ambiguity in calling the constructor: `Book b = new Book("Emma")`.

Is "Emma" the title? The author? **Ambiguous!**

Overloading Methods

We can also overload methods, e.g...

```
public void renew()
public void renew(int days)
```

... same idea; would it be ambiguous? **Not allowed!**

```
public void checkOut()
public boolean checkOut()
```

See Hobbes slides for more examples.

Constructors & Fields

Constructors are often used to initialize fields, e.g.

```
public class Book {

  private String title;
  private String author;
  private int loanDays;          // <-- private!

  public Book(String title, String author) {
    this.title = title;
    this.author = author;
    this.loanDays = 0;
  }
}
```

Visibility

We like to keep these fields private...

```
public class Book {

  private String title;          // <-- private!
  private String author;         // <-- private!
  private int loanDays;          // <-- private!

  public Book(String title, String author) {
    this.title = title;
    this.author = author;
    this.loanDays = 0;
  }
}
```

Accessors

... and gate access! Why? **Encapsulation**. We only want to allow certain actions to be done.

```
public String getTitle() {
  return this.title;
}

public String getAuthor() {
  return this.author;
}
```

Mutators

Allow actions, but supervise what is being done!

```
public void setLoanDays(int loanDays) {
  if (loanDays < 0 || loanDays > 60) {
    // Do nothing, throw error, it's up to you!
  } else {
    this.loanDays = loanDays;
  }
}
```

Accessors & Mutators

These are your gates! How you let data in and out.

Formal	Informal	Example
Accessor	Getter	`public int getDays()`
Mutator	Setter	`public void setDays(int d)`

Guard your gates where appropriate :)

Java Visualizer Example

It's back?! Let's visualize how objects are created from a Java class

Backup link if it's not back!

`toString` lets the object introduce itself

```
public String toString() {
  // Note: Don't sysout here! Let the caller take care of that.
  return this.title + " by " + this.author;
}
```

```
System.out.println(dune.toString());     // either work!
System.out.println(dune);                // implicit toString()
```

Without one you get `Book@6d06d69c`.

Should be annotated with `@Override`, a topic for next week's classes on inheritance.

Eclipse Exercise

TODO 1. Add a `toString` to the `Book` class.

TODO 2. Add various methods to the `Book` class.

Tophat Q1

A check on all of the above.

→ Open Tophat to vote.

Reminder: `static`

Not everything belongs to a book object, some belong to the `Book` class and is shared.

Keyword	Type	Accessed by
`static`	"Class"	`Book.getNumberOfBooks()`
	"Instance"	`b.getTitle()`

We have "Class" and "Instance" fields *and* methods.

Instance and class fields

Instance fields belong to a *particular* book, class fields belong to the `Book` class.

```
public class Book {
  private String author;                          // INSTANCE FIELD
  private String title;                           // INSTANCE FIELD
  private int loanDays;                           // INSTANCE FIELD

  private static final int DEFAULT_LOAN_DAYS = 30;   // CLASS FIELD
  private static int numBooks;                    // CLASS FIELD
}
```

Instance and class methods

Instance methods belong to a *particular* book*, class methods belong to the `Book` class.

```
public class Book {
  public String getTitle() {
    return this.title;
  }

  public static int getNumberOfBooks() {
    return Book.numBooks;
  }
}
```

Instance and class methods

Just like `this` can be used to disambiguate an instance fields and methods, `Book` can be used for class fields and methods.

```
public class Book {
  public String getTitle() {
    return this.title;
  }

  public static int getNumberOfBooks() {
    return Book.numBooks;
  }
}
```

Eclipse Exercise

TODO 3. Add `static` fields and methods to `Book`.

Tophat Q2

A check on `static`.

→ Open Tophat to vote.

Questions?